Spring Framework — Aspect-Oriented Programming (AOP)

Part 2: Proxies, Pointcut Expressions and Types of Advice

These notes follow the supplied lecture transcript and diagrams. They explain the proxy concept, Advice, Pointcuts, and the five Advice annotations.

1. Recap: AOP, Cross-Cutting Concerns and Wrapping

AOP separates cross-cutting concerns from business logic. Business logic belongs to the application's domain, while supporting or infrastructure logic can include logging, security checks and time evaluation.

• **Filters** wrap an incoming raw request.

• **Interceptors** wrap controller execution.

• **AOP** can intercept methods across classes, commonly service-layer methods.

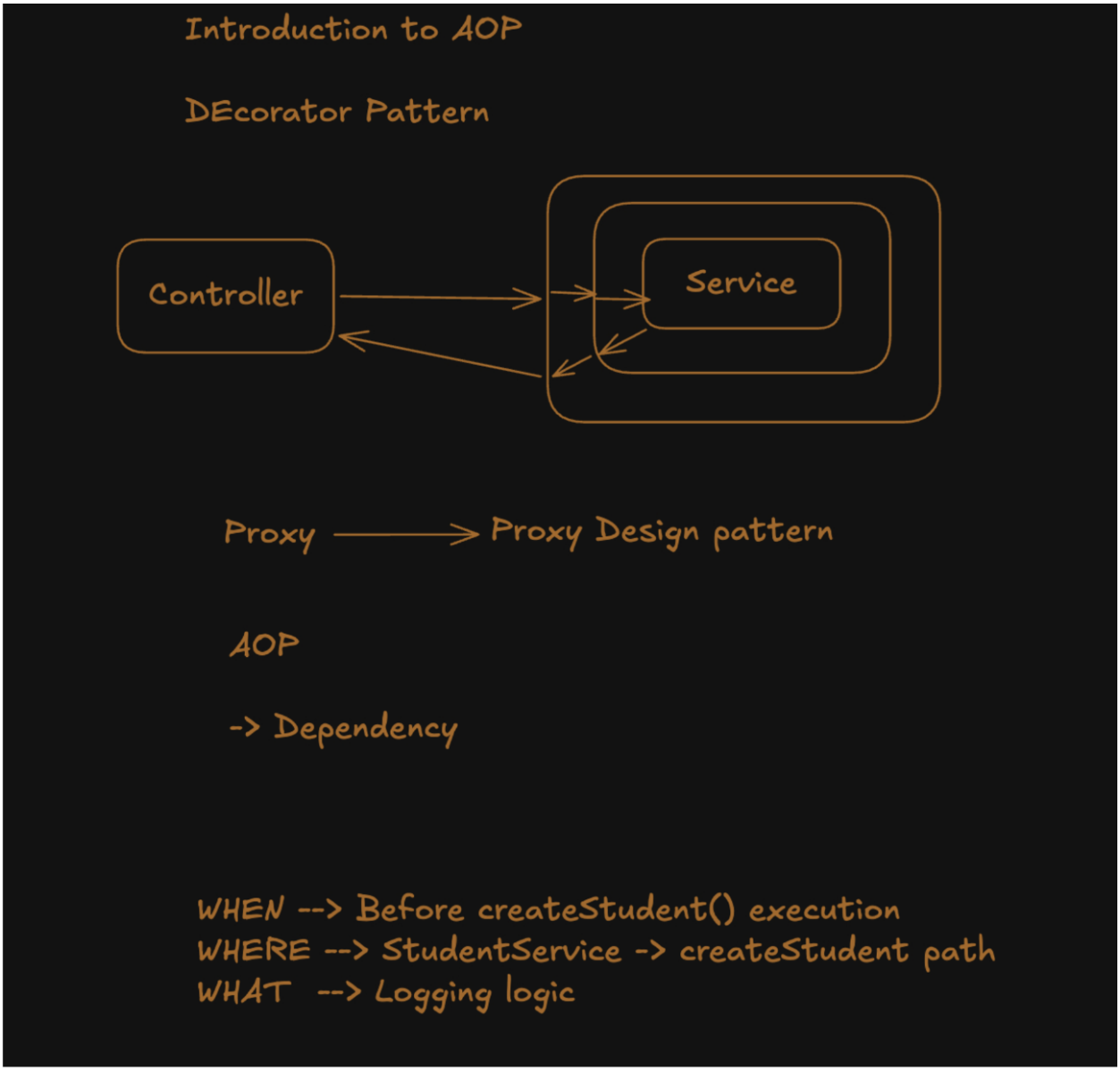
The lecture uses a wrapping model: a Controller interacts with an outer wrapper, which can execute supporting logic and then delegate to the actual Service. Multiple wrappers can be nested.

2. Decorator Pattern vs Proxy Pattern

Decorator Pattern	Proxy Pattern
Adds or enhances functionality around an object.	Controls or mediates access to an object.
Example: adding logging or authentication behavior.	Example: requiring calls to pass through an intermediary.

The two patterns can have a similar structure. Their intent differs: Decorator focuses on adding functionality; Proxy focuses on controlling access. Spring AOP uses proxies internally, while the wrapping behavior can also be understood conceptually using the Decorator Pattern.

Diagram 1 — Proxy / Decorator concept from the lecture



3. Dependency and Example Project Structure

The lecture adds the Spring Boot AOP/AspectJ starter dependency through Maven. The artifact naming can vary by Spring Boot version, so use the starter appropriate to the version of the project.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

The example project contains a StudentController, StudentService and LoggingAspect. The API endpoint used in the demonstration is **POST /api/students**. The Controller calls **studentService.createStudent()**, and the Service prints and returns “Student Saved”.

4. Creating an Aspect Class

An Aspect is a class that contains cross-cutting logic. The lecture uses @Component so Spring manages the class as a bean, and @Aspect to identify it as an Aspect.

```
@Component
@Aspect
public class LoggingAspect {
}
```

• **@Component:** registers the class as a Spring-managed bean.

• **@Aspect:** identifies the class as an Aspect.

5. Three Questions in AOP: WHEN, WHERE and WHAT

Question	Meaning	Example
WHEN	When should the supporting logic execute?	Before method execution.
WHERE	Which method should be intercepted?	StudentService.createStudent().
WHAT	What action should execute?	Logging logic.

6. Advice and Pointcut Expression

Advice describes the action to execute and its timing. **Pointcut** identifies where the Advice should be applied.

```
@Before("execution(* com.example.aopdemo.service.StudentService.createStudent())")
public void logBeforeMethod() {
    System.out.println("Student is going to be saved");
}
```

The expression is a Pointcut Expression. The annotation and the method body together describe the Advice.

7. Reading an execution Pointcut

```
execution(* com.example.aopdemo.service.StudentService.createStudent())
```

Expression part	Meaning
execution	Matches method execution.
*	Any return type.
com.example.aopdemo.service	Package name.
StudentService	Class name.
createStudent()	Method name and parameter list.

8. Important AOP Terminology

Term	Meaning
Aspect	Class containing cross-cutting logic.
Advice	Action executed at a particular point.
Pointcut	Expression identifying where Advice should execute.
Target	Actual object whose method is intercepted.
Proxy	Intermediary object that intercepts calls and delegates to the target.
JoinPoint	Represents an intercepted execution point and provides information about it.

9. How the Proxy Flow Works Conceptually

When AOP is applied, the Controller calls the injected Service bean. Conceptually, the call passes through a proxy, which applies the configured supporting logic and then delegates to the actual target object.

```
StudentController
       |
       v
Student Proxy
       |
       v
Logging Aspect Logic
       |
       v
Actual StudentService
       |
       v
createStudent()
```

The proxy is managed internally by Spring. Applications normally inject and use the Service bean; they do not manually create the proxy.

Diagram 2 — Pointcut, Advice and Proxy relationship



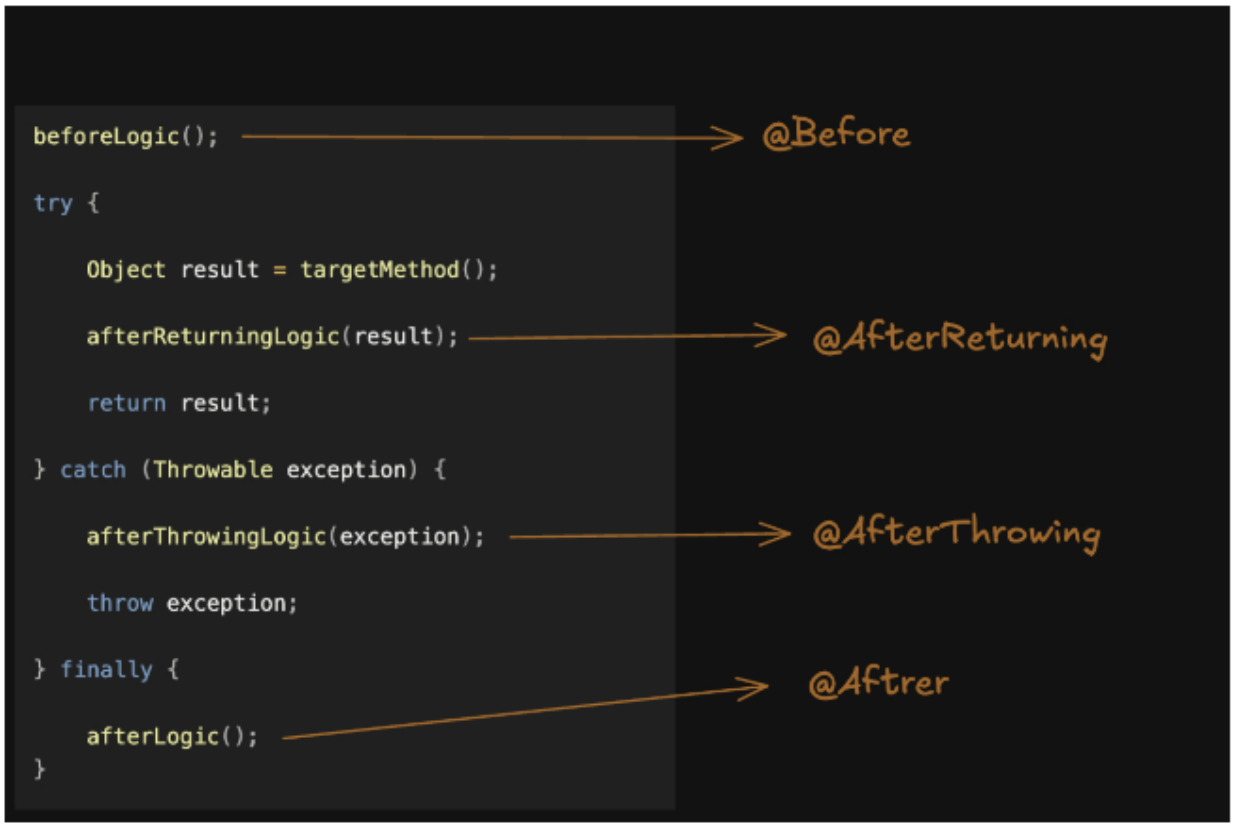
10. Five Types of Advice

The lecture introduces five Advice types. They all describe WHEN the supporting logic should execute.

Advice	Execution timing
@Before	Before the target method starts.
@After	After the target method completes, whether normally or exceptionally.
@AfterReturning	After successful completion and return.
@AfterThrowing	When the target method throws an exception.
@Around	Wraps the target execution and gives control over whether/how it proceeds.

The lecture relates these Advice types to a Java try-catch-finally structure.

Diagram 3 — Advice mapped to try-catch-finally



11. @Before Advice

@Before executes before the target method starts. Common examples in the lecture include logging, authentication checks and precondition validation.

```
@Before("execution(* com.example.aopdemo.service.StudentService.createStudent())")
public void logBeforeMethod() {
    System.out.println("Student is going to be saved");
}
```

A normal return statement inside a @Before method does not stop the target method. Throwing an exception from the Advice can prevent the target method from executing.

```
@Before("execution(* com.example.aopdemo.service.StudentService.createStudent())")
public void validateBeforeMethod() {
    boolean allowed = false;

    if (!allowed) {
        throw new RuntimeException("Method execution not allowed");
    }
}
```

Reading method arguments: JoinPoint provides information about the intercepted method. Its getArgs() method returns the arguments as an Object array.

```
@Before("execution(* com.example.aopdemo.service.StudentService.createStudent(..))")
public void logBeforeMethod(JoinPoint joinPoint) {
    Object[] args = joinPoint.getArgs();
}
```

> **In @Before, arguments can be inspected but not modified through JoinPoint.**

12. @AfterReturning Advice

@AfterReturning executes after the target method completes successfully and returns.

```
@AfterReturning(
    pointcut = "execution(* com.example.aopdemo.service.StudentService.createStudent())",
    returning = "result"
)
public void logAfterReturningMethod(Object result) {
    System.out.println("Returned Value: " + result);
}
```

• Executes only when the target method completes successfully.

• Does not execute when the target method throws an exception.

• The returning attribute allows the returned value to be inspected.

• Use @Around when the returned value needs to be modified.

13. @AfterThrowing Advice

@AfterThrowing executes when the target method throws an exception. It can be used for exception logging and monitoring.

```
@AfterThrowing(
    pointcut = "execution(* com.example.aopdemo.service.StudentService.createStudent())",
    throwing = "exception"
)
public void logAfterThrowingMethod(Exception exception) {
    System.out.println(exception.getMessage());
}
```

• Executes only when the target method throws an exception.

• The throwing attribute exposes the thrown exception to the Advice method.

• It does not replace the target method's exception handling.

14. @After Advice

@After executes after the target method completes, whether it succeeds or throws an exception. The lecture compares it to Java's finally block.

```
@After("execution(* com.example.aopdemo.service.StudentService.createStudent())")
public void logAfterMethod() {
    System.out.println("Method execution completed");
}
```

• Executes in both success and exception cases.

• Useful for completion logging and cleanup.

• Does not consume or suppress the target method's exception.

15. Comparing @After, @AfterReturning and @AfterThrowing

Advice	Successful execution	Exception execution
@After	Executes	Executes
@AfterReturning	Executes	Does not execute
@AfterThrowing	Does not execute	Executes

Memory rule: Success case → @AfterReturning and @After. Exception case → @AfterThrowing and @After.

16. @Around Advice

@Around is the most powerful Advice type discussed in the lecture. It can execute logic before and after the target method and control whether the target method is invoked.

It can also modify arguments, modify the returned value, handle or rethrow exceptions, and invoke the target method through proceed().

ProceedingJoinPoint is used with @Around. Its proceed() method invokes the target method.

```
@Around("execution(* com.example.aopdemo.service.StudentService.createStudent())")
public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
    System.out.println("Before target method");

    Object result = joinPoint.proceed();

    System.out.println("After target method");

    return result;
}
```

> **If proceed() is not called, the target method does not execute. The Advice can return a value directly instead.**

17. JoinPoint vs ProceedingJoinPoint

JoinPoint	ProceedingJoinPoint
Provides information about the intercepted method.	Provides information and control over method execution.
Can access method arguments.	Can access and modify arguments passed to proceed().
Does not provide proceed().	Can invoke the target method using proceed().
Used in @Before, @After, @AfterReturning and @AfterThrowing.	Used in @Around.

```
@Around("execution(* com.example.aopdemo.service.StudentService.createStudent(..))")
public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
    Object[] args = joinPoint.getArgs();

    // Modify arguments here

    Object result = joinPoint.proceed(args);

    return result;
}
```

18. Which Advice Should You Use?

Requirement	Recommended Advice
Log before method execution	@Before
Log after successful execution	@AfterReturning
Log an exception	@AfterThrowing
Execute logic regardless of success or failure	@After
Modify arguments or return value	@Around
Control whether the target method executes	@Around

19. Quick Revision and Interview Questions

AOP terminology

```
Aspect
   |
   v
Advice + Pointcut
   |
   v
Proxy
   |
   v
Target Method
   |
   v
Advice Execution
   |
   v
Result / Exception
```

Five Advice types

Annotation	Remember
@Before	Before the method starts.
@After	Like finally.
@AfterReturning	Successful return only.
@AfterThrowing	Exception only.
@Around	Control around execution.

Interview questions

1. What is an Aspect?
A class containing cross-cutting logic such as logging or security-related behavior.

2. What is Advice?
It defines the action to execute and when it should execute.

3. What is a Pointcut Expression?
It identifies the methods where Advice should be applied.

4. What is a Target object?
The actual object whose method is intercepted.

5. What is a Proxy?
An intermediary object that intercepts method calls and applies configured AOP behavior before delegating to the target.

6. Can @Before stop a method from executing?
A normal return from the Advice cannot stop it. Throwing an exception from the Advice can prevent target execution.

7. Can JoinPoint modify method arguments?
No. JoinPoint can inspect arguments. Use ProceedingJoinPoint in @Around when arguments need to be modified.

8. What happens if proceed() is not called inside @Around?
The target method does not execute.

9. Which Advice executes in both success and exception cases?
@After.

10. Which Advice is used to modify the return value?
@Around.

Final takeaway

> **Spring AOP uses proxies to intercept method calls. An Aspect contains Advice; Advice describes what and when; a Pointcut describes where. Use JoinPoint to inspect an intercepted method and ProceedingJoinPoint in @Around when you need control over execution.**

End of notes — next topic: Pointcut Expressions in depth.